

```

    alloc = p + s - n; // alloc contains the address of the
desired block

    // if the block size matches the requested size, remove
the block from the free list
    if ( s == n ) {

        // if the match is found in the first block update
the pointer of freeblock to point the next of free block
        if ( q == null ) {
            freeblock = next(p);
        } else {
            next(q) = next(p);
        }
    } else {
        size(p) = s - n; // adjust the size of the remaining
free block
    }
}

```

Best Fit

The smallest of the free blocks, whose size is more than or equal to the specified size 'n', is chosen in the Best Fit approach. To discover the best match, this algorithm must run through the full list.

Let's now try to allocate 200 bytes and note the changes in the structure:

0						1023
Address	0	250	550	700	900	950 - 1023
Allocation details	Block 1 allocated	Free 1	Block 2 allocated	Block 3 allocated (new)	Block 4 allocated	Free 2
	Size = 250	Size = 300	Size = 150	Size = 200	Size = 50	Size = 73

The memory Free 1 was ignored, and Free 2 was updated to Block 3. Free 1 is now pointing to Free 3 (since changed to Free 2). As the requested size matches the allocated size, it is removed from the free pointer list.

Worst Fit

The algorithm in the Worst Fit technique always allocates a portion of the largest free memory block. The idea behind this strategy is that by continuously employing a small number of very large blocks to serve the majority of requests, many of the smaller blocks will remain unfragmented.

Let's now try to allocate 200, and see how the structure gets changed.

0							1023
Address	0	250	350	550	700	900	950 - 1023
Allocation details	Block 1 allocated	Free 1	Block 2 allocated (new)	Block 3 allocated	Free 2	Block 4 allocated	Free 3
	Size = 250	Size = 100	Size = 200	Size = 150	Size = 200	Size = 50	Size = 73

As Free 1 holds the maximum of 300 bytes, the memory is allocated there. If we try to allocate 100, though we have the first free size with 100 bytes, it will still be allocated to Free 2, which is the maximum now.

Each method has its own advantages. Let's understand that with examples.

First Fit as best case

In the scenario given below, only First Fit is able to serve all the requests.

Request	Blocks remaining using		
	First Fit	Best Fit	Worst Fit
Initially	110, 54	110, 54	110, 54
25	85, 54	110, 54	85, 54
70	15, 54	40, 29	15, 54
50	15, 4	Cannot be fulfilled	15, 4
14	1, 4		1, 4
1	0, 4		1, 3
2	0, 2		Cannot be fulfilled

Best Fit as best case

In the scenario given below, only Best Fit is able to serve all the requests.

Request	Blocks remaining using		
	First Fit	Best Fit	Worst Fit
Initially	110, 54	110, 54	110, 54
50	60, 54	110, 4	60, 54
100	Cannot be fulfilled	10, 4	Cannot be fulfilled

Worst Fit as best case

Only Worst Fit is able to serve all the requests in the instances given below.

Request	Blocks remaining using		
	First Fit	Best Fit	Worst Fit
Initially	200, 300, 100	200, 300, 100	200, 300, 100
150	50, 300, 100	50, 300, 100	200, 150, 100
100	50, 200, 100	50, 300, 0	100, 150, 100
125	50, 75, 100	50, 175, 0	100, 50, 100
100	50, 75, 0	50, 75, 0	0, 50, 100
100	Cannot be fulfilled	Cannot be fulfilled	0, 50, 0

As mentioned earlier, each method has its uses though, generally, the First Fit method is preferred. **END** 🐧

Reference

'Data Structures using C and C++' by Yedidyah Langsam, Moshe J. Augenstein, Aaron M. Tenenbanum

By: Thangaselvi Arichandrapandian

The author is employed in a leading bank as an AVP. She is a perennial learner who loves the quote:

"The more I learn, the more I realise how much I don't know."
- Albert Einstein